JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class NumberFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

Direct Known Subclasses:

`ChoiceFormat`, `CompactNumberFormat`, `DecimalFormat`

```
public abstract class NumberFormat
extends Format
```

`NumberFormat` is the abstract base class for all number formats. This class provides the interface for formatting and parsing numbers in a localized manner. This enables code that can be completely independent of the locale conventions for decimal points, thousands-separators, the particular decimal digits used, or whether the number format is even decimal. For example, this class could be used within an application to produce a number in a currency format according to the conventions of the desired locale.

Getting a NumberFormat

To get a `NumberFormat` for the default Locale, use one of the static factory methods that return a concrete subclass of `NumberFormat`. The following formats all provide an example of formatting the `Number` "2000.50" with the `US` locale as the default locale.

- Use `getInstance()` or `getNumberInstance()` to get a decimal format. For example, `"2,000.5"`.
- Use `getIntegerInstance()` to get an integer number format. For example, `"2,000"`.
- Use `getCurrencyInstance()` to get a currency number format. For example, `"$2,000.50"`.
- Use `getCompactNumberInstance()` to get a compact number format. For example, `"2K"`.
- Use `getPercentInstance()` to get a format for displaying percentages. For example, `"200,050%"`.

Alternatively, if a `NumberFormat` for a different locale is required, use one of the overloaded factory methods that take `Locale` as a parameter, for example, `getIntegerInstance(Locale)`. If the installed locale-sensitive service implementation does not support the given `Locale`, the parent locale chain will be looked up, and a `Locale` used that is supported.

Locale Extensions

Formatting behavior can be changed when using a locale that contains any of the following Unicode extensions,

- "nu" (Numbering System) - Overrides the decimal digits used
- "rg" (Region Override) - Overrides the country used
- "cf" (Currency Format style)
  - Overrides the Currency Format style used
- "cu" (Currency Type) - Overrides the Currency used

For both "nu" and "cu", if they are specified in addition to "rg", the respective values from the "nu" and "cu" extension supersede the implicit ones from the "rg" extension. Although Unicode extensions defines various keys and values, actual locale-sensitive service implementations in a Java Runtime Environment might not support any particular Unicode locale attributes or key/type pairs.

Below is an example of a "US" locale currency format with accounting style,

```
NumberFormat.getCurrencyInstance(Locale.forLanguageTag("en-US-u-cf-account"));
```

With this style, a negative value is formatted enclosed in parentheses, instead of being prepended with a minus sign.

Using NumberFormat

The following is an example of formatting and parsing in a localized fashion,

```
NumberFormat currencyFormat = NumberFormat.getCurrencyInstance(Locale.US);
currencyFormat.format(100000); // returns "$100,000.00"
currencyFormat.parse("$100,000.00"); // returns 100000
```

Customizing NumberFormat

`NumberFormat` provides API to customize formatting and parsing behavior,

- `setParseIntegerOnly(boolean)`; when `true`, will only return the integer portion of the number parsed from the String.
- `setMinimumFractionDigits(int)`; Use to adjust the expected digits when formatting. Use any of the other minimum/maximum or fraction/integer setter methods in the same manner. These methods have no impact on parsing behavior.
- `setGroupingUsed(boolean)`; when `true`, formatted numbers will be displayed with grouping separators. Additionally, when `false`, parsing will not expect grouping separators in the parsed String.

- `setStrict(boolean)`; when `true`, parsing will be done strictly. The behavior of strict parsing should be referred to in the implementing `NumberFormat` subclass.

To provide more control over formatting or parsing behavior, type checking can be done to safely convert to an implementing subclass of `NumberFormat`; this provides additional methods defined by the subclass. For example,

```
NumberFormat nFmt = NumberFormat.getInstance(Locale.US);
if (nFmt instanceof DecimalFormat dFmt) {
    dFmt.setDecimalSeparatorAlwaysShown(true);
    dFmt.format(100); // returns "100."
}
```

The `NumberFormat` subclass returned by the factory methods is dependent on the locale-service provider implementation installed, and may not always be `DecimalFormat` or `CompactNumberFormat`.

You can also use forms of the `parse` and `format` methods with `ParsePosition` and `FieldPosition` to allow you to:

- Progressively parse through pieces of a string
- Align the decimal point and other areas

For example, you can align numbers in two ways:

1. If you are using a monospaced font with spacing for alignment, you can pass the `FieldPosition` in your format call, with `field = INTEGER_FIELD`. On output, `getEndIndex` will be set to the offset between the last character of the integer and the decimal. Add (desiredSpaceCount - getEndIndex) spaces at the front of the string.
2. If you are using proportional fonts, instead of padding with spaces, measure the width of the string in pixels from the start to `getEndIndex`. Then move the pen by (desiredPixelWidth - widthToAlignmentPoint) before drawing the text. It also works where there is no decimal, but possibly additional characters at the end, e.g., with parentheses in negative numbers: "(12)" for -12.

Leniency

`NumberFormat` by default, parses leniently. Subclasses may consider implementing strict parsing and as such, overriding and providing implementations for the optional `isStrict()` and `setStrict(boolean)` methods.

Lenient parsing should be used when attempting to parse a number out of a String that contains non-numerical or non-format related values. For example, using a `Locale.US` currency format to parse the number `1000` out of the String "$1,000.00 was paid". Lenient parsing also allows loose matching of characters in the source text. For example, an implementation of the `NumberFormat` class may allow matching "−" (U+2212 MINUS SIGN) to the "-" (U+002D HYPHEN-MINUS) pattern character when used as a negative prefix.

Strict parsing should be used when attempting to ensure a String adheres exactly to a locale's conventions, and can thus serve to validate input. For example, successfully parsing the number `1000.55` out of the String "1.000,55" confirms the String exactly adhered to the `Locale.GERMANY` numerical conventions.

Synchronization

Number formats are generally not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

Implementation Requirements:

Null Parameter Handling

- The `format(double, StringBuffer, FieldPosition)`, `format(long, StringBuffer, FieldPosition)` and `parse(String, ParsePosition)` methods may throw `NullPointerException`, if any of their parameter is `null`. The subclass may provide its own implementation and specification about `NullPointerException`.

Default RoundingMode

- The default implementation provides rounding modes defined in `RoundingMode` for formatting numbers. It uses the round half-even algorithm. To change the rounding mode use `setRoundingMode`. The `NumberFormat` returned by the static factory methods is configured to round floating point numbers using half-even rounding (see `RoundingMode.HALF_EVEN`) for formatting.

Since:

1.1

External Specifications

Unicode Locale Data Markup Language (LDML)

See Also:

DecimalFormat, ChoiceFormat, CompactNumberFormat, Locale, Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	NumberFormat.Field	Defines constants that are used as attribute keys in the AttributedCharacterIterator returned from NumberFormat.formatToCharacterIterator and as field identifiers in FieldPosition.
static enum	NumberFormat.Style	A number format style.

Field Summary

Fields

Modifier and Type	Field	Description
static final int	FRACTION_FIELD	Field constant used to construct a FieldPosition object.
static final int	INTEGER_FIELD	Field constant used to construct a FieldPosition object.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	NumberFormat()	Sole constructor.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
Object	**clone**()	Overrides Cloneable.
boolean	**equals**(Object obj)	Compares the specified object with this NumberFormat for equality.
final String	**format**(double number)	Specialization of format.
abstract StringBuffer	**format**(double number, StringBuffer toAppendTo, FieldPosition pos)	Specialization of format.
final String	**format**(long number)	Specialization of format.
abstract StringBuffer	**format**(long number, StringBuffer toAppendTo, FieldPosition pos)	Specialization of format.
StringBuffer	**format**(Object number, StringBuffer toAppendTo, FieldPosition pos)	Formats a number and appends the resulting text to the given string buffer.
static Locale[]	**getAvailableLocales**()	Returns an array of all locales for which the get*Instance methods of this class can return localized instances.
static NumberFormat	**getCompactNumberInstance**	Returns a compact number format for the default FORMAT locale with "SHORT" format style.
static NumberFormat	**getCompactNumberInstance** (Locale locale, NumberFormat.Style forma	Returns a compact number format for the specified locale and formatStyle.
Currency	**getCurrency**()	

		Gets the currency used by this number format when formatting currency values.
static final NumberFormat	**getCurrencyInstance**()	Returns a currency format for the current default FORMAT locale.
static NumberFormat	**getCurrencyInstance** (Locale inLocale)	Returns a currency format for the specified locale.
static final NumberFormat	**getInstance**()	Returns a general-purpose number format for the current default FORMAT locale.
static NumberFormat	**getInstance** (Locale inLocale)	Returns a general-purpose number format for the specified locale.
static final NumberFormat	**getIntegerInstance**()	Returns an integer number format for the current default FORMAT locale.
static NumberFormat	**getIntegerInstance** (Locale inLocale)	Returns an integer number format for the specified locale.
int	**getMaximumFractionDigits**	Returns the maximum number of digits allowed in the fraction portion of a number during formatting.
int	**getMaximumIntegerDigits**(	Returns the maximum number of digits allowed in the integer portion of a number during formatting.
int	**getMinimumFractionDigits**	Returns the minimum number of digits allowed in the fraction portion of

		a number during formatting.
int	**getMinimumIntegerDigits**(	Returns the minimum number of digits allowed in the integer portion of a number during formatting.
static final NumberFormat	**getNumberInstance**()	Returns a general-purpose number format for the current default FORMAT locale.
static NumberFormat	**getNumberInstance** (Locale inLocale)	Returns a general-purpose number format for the specified locale.
static final NumberFormat	**getPercentInstance**()	Returns a percentage format for the current default FORMAT locale.
static NumberFormat	**getPercentInstance** (Locale inLocale)	Returns a percentage format for the specified locale.
RoundingMode	**getRoundingMode**()	Gets the RoundingMode used in this NumberFormat.
int	**hashCode**()	Returns the hash code for this NumberFormat.
boolean	**isGroupingUsed**()	Returns true if grouping is used in this format.
boolean	**isParseIntegerOnly**()	Returns true if this format will parse numbers as integers only.
boolean	**isStrict**()	Returns true if this format will parse

		numbers strictly; `false` otherwise.
`Number`	**`parse`**(`String source`)	Parses text from the beginning of the given string to produce a `Number`.
`abstract Number`	**`parse`**(`String source, ParsePosition parsePosit`	Parses text from the beginning of the given string to produce a `Number`.
`final Object`	**`parseObject`** (`String source, ParsePosition pos`)	Parses text from the given string to produce an object.
`void`	**`setCurrency`** (`Currency currency`)	Sets the currency used by this number format when formatting currency values.
`void`	**`setGroupingUsed`** (`boolean newValue`)	Set whether or not grouping will be used in this format.
`void`	**`setMaximumFractionDigits`** (`int newValue`)	Sets the maximum number of digits allowed in the fraction portion of a number during formatting.
`void`	**`setMaximumIntegerDigits`** (`int newValue`)	Sets the maximum number of digits allowed in the integer portion of a number during formatting.
`void`	**`setMinimumFractionDigits`** (`int newValue`)	Sets the minimum number of digits allowed in the fraction portion of a number during formatting.

void	**setMinimumIntegerDigits**(int newValue)	Sets the minimum number of digits allowed in the integer portion of a number during formatting.
void	**setParseIntegerOnly**(boolean value)	Sets whether or not numbers should be parsed as integers only.
void	**setRoundingMode**(RoundingMode roundingMo	Sets the RoundingMode used in this NumberFormat.
void	**setStrict**(boolean strict)	Change the leniency value for parsing.

Methods declared in class Format

format, formatToCharacterIterator, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Field Details

INTEGER_FIELD

```
public static final int INTEGER_FIELD
```

Field constant used to construct a FieldPosition object. Signifies that the position of the integer part of a formatted number should be returned.

See Also:

FieldPosition, Constant Field Values

FRACTION_FIELD

```
public static final int FRACTION_FIELD
```

Field constant used to construct a FieldPosition object. Signifies that the position of the fraction part of a formatted number should be returned.

See Also:

FieldPosition, Constant Field Values

Constructor Details

NumberFormat

```
protected NumberFormat()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

format

```
public StringBuffer format(Object number,
                           StringBuffer toAppendTo,
                           FieldPosition pos)
```

Formats a number and appends the resulting text to the given string buffer. The number can be of any subclass of `Number`.

This implementation extracts the number's value using `Number.longValue()` for all integral type values that can be converted to `long` without loss of information, including `BigInteger` values with a `bit length` of less than 64, and `Number.doubleValue()` for all other types. It then calls `format(long,java.lang.StringBuffer,java.text.FieldPosition)` or `format(double,java.lang.StringBuffer,java.text.FieldPosition)`. This may result in loss of magnitude information and precision for `BigInteger` and `BigDecimal` values.

Specified by:

`format` in class `Format`

Parameters:

`number` - the number to format

`toAppendTo` - the `StringBuffer` to which the formatted text is to be appended

`pos` - keeps track on the position of the field within the returned string. For example, for formatting a number `1234567.89` in `Locale.US` locale, if the given `fieldPosition` is `INTEGER_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 9, respectively for the output string `1,234,567.89`.

Returns:

the value passed in as `toAppendTo`

Throws:

`IllegalArgumentException` - if `number` is null or not an instance of `Number`.

NullPointerException - if toAppendTo or pos is null

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

FieldPosition

parseObject

```
public final Object parseObject(String source,
                                ParsePosition pos)
```

Parses text from the given string to produce an object.

This method attempts to parse text starting at the index given by pos. If parsing succeeds, then the index of pos is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed object is returned. The updated pos can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of pos is not changed, the error index of pos is set to the index of the character where the error occurred, and null is returned.

Specified by:

parseObject in class Format

Implementation Requirements:

This implementation is equivalent to calling parse(source, pos).

Parameters:

source - the String to parse

pos - A ParsePosition object with index and error index information as described above.

Returns:

A Number parsed from the string. In case of error, returns null.

Throws:

NullPointerException - if source or pos is null.

format

```
public final String format(double number)
```

Specialization of format.

Parameters:

number - the double number to format

Returns:

the formatted String

Throws:

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

`Format.format(Object)`

format

```
public final String format(long number)
```

Specialization of format.

Parameters:

`number` - the long number to format

Returns:

the formatted String

Throws:

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

`Format.format(Object)`

format

```
public abstract StringBuffer format(double number,
                                    StringBuffer toAppendTo,
                                    FieldPosition pos)
```

Specialization of format.

Parameters:

`number` - the double number to format

`toAppendTo` - the StringBuffer to which the formatted text is to be appended

`pos` - keeps track on the position of the field within the returned string. For example, for formatting a number `1234567.89` in `Locale.US` locale, if the given `fieldPosition` is `INTEGER_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 9, respectively for the output string `1,234,567.89`.

Returns:

the formatted StringBuffer

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(Object)

format

```
public abstract StringBuffer format(long number,
                                    StringBuffer toAppendTo,
                                    FieldPosition pos)
```

Specialization of format.

Parameters:

number - the long number to format

toAppendTo - the StringBuffer to which the formatted text is to be appended

pos - keeps track on the position of the field within the returned string. For example, for formatting a number 123456789 in Locale.US locale, if the given fieldPosition is INTEGER_FIELD, the begin index and end index of fieldPosition will be set to 0 and 11, respectively for the output string 123,456,789.

Returns:

the formatted StringBuffer

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(Object)

parse

```
public abstract Number parse(String source,
                             ParsePosition parsePosition)
```

Parses text from the beginning of the given string to produce a Number.

This method attempts to parse text starting at the index given by the ParsePosition. If parsing succeeds, then the index of the ParsePosition is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed number is returned. The updated ParsePosition can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of the ParsePosition is not changed, the error index of the

ParsePosition is set to the index of the character where the error occurred, and null is returned.

This method will return a Long if possible (e.g., within the range [Long.MIN_VALUE, Long.MAX_VALUE] and with no decimals), otherwise a Double.

Parameters:

source - the String to parse

parsePosition - A ParsePosition object with index and error index information as described above.

Returns:

A Number parsed from the string. In case of failure, returns null.

Throws:

NullPointerException - if source or ParsePosition is null.

See Also:

isStrict()

parse

```
public Number parse(String source)
             throws ParseException
```

Parses text from the beginning of the given string to produce a Number.

This method will return a Long if possible (e.g., within the range [Long.MIN_VALUE, Long.MAX_VALUE] and with no decimals), otherwise a Double.

Parameters:

source - A String, to be parsed from the beginning.

Returns:

A Number parsed from the string.

Throws:

ParseException - if parsing fails

NullPointerException - if source is null.

See Also:

isStrict()

isParseIntegerOnly

```
public boolean isParseIntegerOnly()
```

Returns `true` if this format will parse numbers as integers only. The `ParsePosition` index will be set to the position of the decimal symbol. The exact format accepted by the parse operation is locale dependent. For example in the English locale, with ParseIntegerOnly true, the string "123.45" would be parsed as the integer value 123.

Returns:

`true` if numbers should be parsed as integers only; `false` otherwise

setParseIntegerOnly

```
public void setParseIntegerOnly(boolean value)
```

Sets whether or not numbers should be parsed as integers only.

Parameters:

`value` - `true` if numbers should be parsed as integers only; `false` otherwise

See Also:

`isParseIntegerOnly()`

isStrict

```
public boolean isStrict()
```

Returns `true` if this format will parse numbers strictly; `false` otherwise.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses should override this method when implementing strict parsing.

Returns:

`true` if this format will parse numbers strictly; `false` otherwise

Throws:

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

23

See Also:

Leniency Section, `setStrict(boolean)`

setStrict

```
public void setStrict(boolean strict)
```

Change the leniency value for parsing. Parsing can either be strict or lenient, by default it is lenient.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses should override this method when implementing strict parsing.

Parameters:

`strict` - `true` if parsing should be done strictly; `false` otherwise

Throws:

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

23

See Also:

Leniency Section, `isStrict()`

getInstance

```
public static final NumberFormat getInstance()
```

Returns a general-purpose number format for the current default `FORMAT` locale. This is the same as calling `getNumberInstance()`.

Returns:

the `NumberFormat` instance for general-purpose number formatting

getInstance

```
public static NumberFormat getInstance(Locale inLocale)
```

Returns a general-purpose number format for the specified locale. This is the same as calling `getNumberInstance(inLocale)`.

Parameters:

`inLocale` - the desired locale

Returns:

the `NumberFormat` instance for general-purpose number formatting

getNumberInstance

```
public static final NumberFormat getNumberInstance()
```

Returns a general-purpose number format for the current default `FORMAT` locale.

This is equivalent to calling
`getNumberInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

the `NumberFormat` instance for general-purpose number formatting

See Also:

`Locale.getDefault(java.util.Locale.Category)`,
`Locale.Category.FORMAT`

getNumberInstance

```
public static NumberFormat getNumberInstance(Locale inLocale)
```

Returns a general-purpose number format for the specified locale.

Parameters:

`inLocale` - the desired locale

Returns:

the `NumberFormat` instance for general-purpose number formatting

getIntegerInstance

```
public static final NumberFormat getIntegerInstance()
```

Returns an integer number format for the current default `FORMAT` locale. The returned number format is configured to round floating point numbers to the nearest integer using half-even rounding (see `RoundingMode.HALF_EVEN`) for formatting, and to parse only the integer part of an input string (see `isParseIntegerOnly`).

This is equivalent to calling
`getIntegerInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a number format for integer values

Since:

1.4

See Also:

`getRoundingMode()`,
`Locale.getDefault(java.util.Locale.Category)`,
`Locale.Category.FORMAT`

getIntegerInstance

```
public static NumberFormat getIntegerInstance(Locale inLocale)
```

Returns an integer number format for the specified locale. The returned number format is configured to round floating point numbers to the nearest integer using half-even rounding (see `RoundingMode.HALF_EVEN`) for formatting, and to parse only the integer part of an input string (see `isParseIntegerOnly`).

Parameters:

`inLocale` - the desired locale

Returns:

a number format for integer values

Since:

1.4

See Also:

`getRoundingMode()`

getCurrencyInstance

```
public static final NumberFormat getCurrencyInstance()
```

Returns a currency format for the current default `FORMAT` locale.

This is equivalent to calling `getCurrencyInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

the `NumberFormat` instance for currency formatting

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getCurrencyInstance

```
public static NumberFormat getCurrencyInstance(Locale inLocale)
```

Returns a currency format for the specified locale.

If the specified locale contains the "`cf`" (currency format style) Unicode extension, the returned currency format uses the style if it is available. Otherwise, the style uses the default "`standard`" currency format. For example, if the style designates "`account`", negative currency amounts use a pair of parentheses in some locales.

Parameters:

`inLocale` - the desired locale

Returns:

the NumberFormat instance for currency formatting

External Specifications

Unicode Locale Data Markup Language (LDML)

getPercentInstance

```
public static final NumberFormat getPercentInstance()
```

Returns a percentage format for the current default FORMAT locale.

This is equivalent to calling
getPercentInstance(Locale.getDefault(Locale.Category.FORMAT)).

Returns:

the NumberFormat instance for percentage formatting

See Also:

Locale.getDefault(java.util.Locale.Category),
Locale.Category.FORMAT

getPercentInstance

```
public static NumberFormat getPercentInstance(Locale inLocale)
```

Returns a percentage format for the specified locale.

Parameters:

inLocale - the desired locale

Returns:

the NumberFormat instance for percentage formatting

getCompactNumberInstance

```
public static NumberFormat getCompactNumberInstance()
```

Returns a compact number format for the default FORMAT locale with "SHORT" format style.

Returns:

A NumberFormat instance for compact number formatting

Since:

12

See Also:

CompactNumberFormat,
NumberFormat.Style,
Locale.getDefault(java.util.Locale.Category),

Locale.Category.FORMAT

getCompactNumberInstance

```
public static NumberFormat getCompactNumberInstance
(Locale locale,
 NumberFormat.Style formatStyle)
```

Returns a compact number format for the specified `locale` and `formatStyle`.

Parameters:

`locale` - the desired locale

`formatStyle` - the style for formatting a number

Returns:

A `NumberFormat` instance for compact number formatting

Throws:

`NullPointerException` - if `locale` or `formatStyle` is `null`

Since:

12

See Also:

`CompactNumberFormat`, `NumberFormat.Style`, `Locale`

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `get*Instance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `NumberFormatProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

Returns:

An array of locales for which localized `NumberFormat` instances are available.

hashCode

```
public int hashCode()
```

Returns the hash code for this `NumberFormat`.

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

This method calculates the hash code value using the values returned by `getMaximumIntegerDigits()` and `getMaximumFractionDigits()`.

Returns:

the hash code for this `NumberFormat`

See Also:

`Object.hashCode()`

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this `NumberFormat` for equality. Returns true if the object is also a `NumberFormat` and the two formats would format any value the same.

Overrides:

`equals` in class `Object`

Implementation Requirements:

This method performs an equality check with a notion of class identity based on `getClass()`, rather than `instanceof`. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `NumberFormat`

See Also:

`Object.equals(Object)`

clone

```
public Object clone()
```

Overrides Cloneable.

Overrides:

`clone` in class `Format`

Returns:

a clone of this instance.

See Also:

`Cloneable`

isGroupingUsed

```
public boolean isGroupingUsed()
```

Returns true if grouping is used in this format. This applies to both formatting and parsing. The grouping separator as well as the size of each group is locale dependent and is determined by sub-classes of NumberFormat. For example, consider a `NumberFormat` that expects a `","` grouping separator symbol with a grouping size of 3.

- Formatting `1234567` with grouping on returns `"1,234,567"`
- Parsing `"1,234,567"` with grouping off returns `1`
- Parsing `"1,234,567"` with grouping off when `isStrict()` returns `true` throws `ParseException`

Returns:

`true` if grouping is used; `false` otherwise

See Also:

`setGroupingUsed(boolean)`, Leniency Section

setGroupingUsed

```
public void setGroupingUsed(boolean newValue)
```

Set whether or not grouping will be used in this format. This applies to both formatting and parsing.

Parameters:

`newValue` - `true` if grouping is used; `false` otherwise

See Also:

`isGroupingUsed()`

getMaximumIntegerDigits

```
public int getMaximumIntegerDigits()
```

Returns the maximum number of digits allowed in the integer portion of a number during formatting.

Returns:

the maximum number of digits

See Also:

`setMaximumIntegerDigits(int)`

setMaximumIntegerDigits

```
public void setMaximumIntegerDigits(int newValue)
```

Sets the maximum number of digits allowed in the integer portion of a number during formatting. `maximumIntegerDigits` must be ≥ `minimumIntegerDigits`. If the new value for `maximumIntegerDigits` is less than the current value of `minimumIntegerDigits`, then `minimumIntegerDigits` will also be set to the new value. Negative input values are replaced with 0.

Parameters:

`newValue` - the maximum number of integer digits to be shown. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

`getMaximumIntegerDigits()`

getMinimumIntegerDigits

```
public int getMinimumIntegerDigits()
```

Returns the minimum number of digits allowed in the integer portion of a number during formatting.

Returns:

the minimum number of digits

See Also:

`setMinimumIntegerDigits(int)`

setMinimumIntegerDigits

```
public void setMinimumIntegerDigits(int newValue)
```

Sets the minimum number of digits allowed in the integer portion of a number during formatting. `minimumIntegerDigits` must be ≤ `maximumIntegerDigits`. If the new value for `minimumIntegerDigits` exceeds the current value of `maximumIntegerDigits`, then `maximumIntegerDigits` will also be set to the new value. Negative input values are replaced with 0.

Parameters:

`newValue` - the minimum number of integer digits to be shown. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

`getMinimumIntegerDigits()`

getMaximumFractionDigits

```
public int getMaximumFractionDigits()
```

Returns the maximum number of digits allowed in the fraction portion of a number during formatting.

Returns:

the maximum number of digits.

See Also:

`setMaximumFractionDigits(int)`

setMaximumFractionDigits

```
public void setMaximumFractionDigits(int newValue)
```

Sets the maximum number of digits allowed in the fraction portion of a number during formatting. `maximumFractionDigits` must be ≥ `minimumFractionDigits`. If the new value for `maximumFractionDigits` is less than the current value of `minimumFractionDigits`, then `minimumFractionDigits` will also be set to the new value. Negative input values are replaced with 0.

Parameters:

`newValue` - the maximum number of fraction digits to be shown. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

`getMaximumFractionDigits()`

getMinimumFractionDigits

```
public int getMinimumFractionDigits()
```

Returns the minimum number of digits allowed in the fraction portion of a number during formatting.

Returns:

the minimum number of digits

See Also:

`setMinimumFractionDigits(int)`

setMinimumFractionDigits

```
public void setMinimumFractionDigits(int newValue)
```

Sets the minimum number of digits allowed in the fraction portion of a number during formatting. `minimumFractionDigits` must be ≤ `maximumFractionDigits`. If the new value for `minimumFractionDigits` exceeds

the current value of `maximumFractionDigits`, then `maximumFractionDigits` will also be set to the new value. Negative input values are replaced with 0.

Parameters:

`newValue` - the minimum number of fraction digits to be shown. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

`getMinimumFractionDigits()`

getCurrency

```
public Currency getCurrency()
```

Gets the currency used by this number format when formatting currency values. The initial value is derived in a locale dependent way. The returned value may be `null` if no valid currency could be determined and no currency has been set using `setCurrency(Currency)`.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses should override this method if currency formatting is desired.

Returns:

the currency used by this number format, or `null`

Throws:

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

1.4

setCurrency

```
public void setCurrency(Currency currency)
```

Sets the currency used by this number format when formatting currency values. This does not update the minimum or maximum number of fraction digits used by the number format.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses should override this method if currency formatting is desired.

Parameters:

`currency` - the new currency to be used by this number format

Throws:

`NullPointerException` - if `currency` is `null`

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

1.4

getRoundingMode

`public RoundingMode getRoundingMode()`

Gets the `RoundingMode` used in this NumberFormat.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses which handle different rounding modes should override this method.

Returns:

The `RoundingMode` used for this NumberFormat.

Throws:

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

1.6

See Also:

`setRoundingMode(RoundingMode)`

setRoundingMode

`public void setRoundingMode(RoundingMode roundingMode)`

Sets the `RoundingMode` used in this NumberFormat.

Implementation Requirements:

The default implementation always throws `UnsupportedOperationException`. Subclasses which handle different rounding modes should override this method.

Parameters:

`roundingMode` - The `RoundingMode` to be used

Throws:

`NullPointerException` - if `roundingMode` is `null`

`UnsupportedOperationException` - if the implementation of this method does not support this operation

Since:

1.6

See Also:

`getRoundingMode()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.